

Module 2

Understanding the Ethereum Virtual Machine

What Is the EVM?

The CPU of Ethereum's Global Computer

The Ethereum Virtual Machine (EVM) is a decentralised, Turing-complete software engine that executes smart contracts and processes transactions across all Ethereum nodes.

Turing-complete meaning it can solve any computational problem that any other computer can, provided it has enough time and memory

It acts as a global, virtual computer that ensures every node agrees on the exact same state changes after executing code, providing a secure, **trustless** environment

Thousands of identical CPUs running in parallel, each executes the same bytecode and each produces the same result.
Recap: This is what's known as determinism.

Ethereum = State Transition System

New State = $f(\text{Current State, Transaction})$

EVM is the engine that computes $f()$

Key Property:

Every node reaches the same result independently

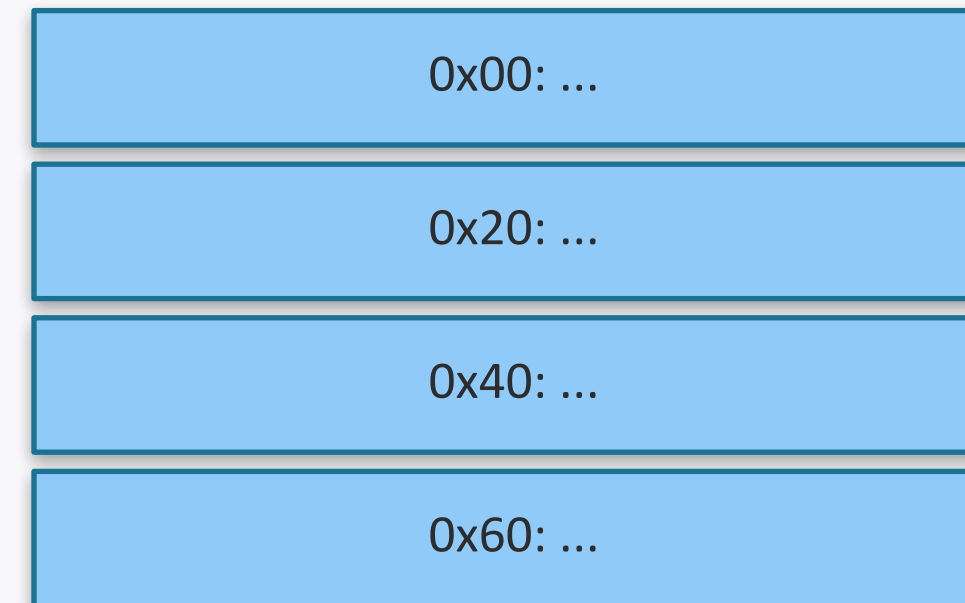
No central authority needed for consensus

EVM Architecture: Stack, Memory, Storage

Stack (LIFO)



Memory (Temp)



Storage (Perm)



Stack:

- The EVM is a, "stack-based" virtual machine architecture that processes smart contracts using a Last-In-First-Out (LIFO) data structure. It temporarily stores 256-bit words (max 1024 items) to execute opcodes like PUSH and POP for transactions.

Memory:

- A volatile, byte-addressable, and temporary data location used by the EVM to store data during smart contract execution. It stores intermediate calculations, arrays, and strings, and it is cleared after each function call.

Storage:

- Refers to the persistent, long-term database where smart contracts hold their state variables. It remains intact between transactions and function calls. It acts as the contract's "memory," storing data permanently on the blockchain for high costs

Comparison Table

	Storage	Memory	Stack
Persistence	Permanent	Temporary	Temporary (Per Call)
Cost	Expensive	Low	Almost Free
Size	Effectively Infinite	Limited. Cost increases with data stored	Max 1024 elements
Access	SLOAD/SSTORE	MLOAD/MSTORE	PUSH/POP

The Stack (256-bit words)

A LIFO (Last In, First Out) data structure

Max depth: 1,024 items

Most operations pop values and push results

Example Stack Operations:

PUSH 2 → [2]

PUSH 3 → [2, 3]

ADD → [5] (pops both, pushes result)

Why a Stack?

Simple and fast for bytecode execution

Natural fit for EVM instruction set

Memory (Temporary Storage)

A temporary whiteboard for execution

Erased after function execution

Cheaper than storage

Larger than stack (no hard limit)

Common Uses:

Store function arguments (strings and arrays need to be stored in memory)

Intermediate calculations (holding cached results)

Building return values

Key Property:

Not part of global state

Each call gets fresh memory

Storage (Permanent On-Chain)

A persistent filing cabinet for state

Lives on-chain forever

Part of Ethereum's global state

Organized in 32-byte slots

Example (Solidity):

Contract counter {

uint256 public count; ← stored in slot 0

address public owner; ← stored in slot 1

}

Why It's Expensive:

SSTORE (write) $\approx$ 20,000 gas

Most gas optimisation is about reducing writes

Calldata (Read-Only Input)

This input data passed into a function

Read-only during execution

Cannot be modified

Cheaper to read than memory

Typical Structure:

First 4 bytes: function selector

Remaining bytes: encoded arguments

```
function transfer(address _to, uint256 _amount)
```

Recipient (to): 0x123456789012345678901234567890

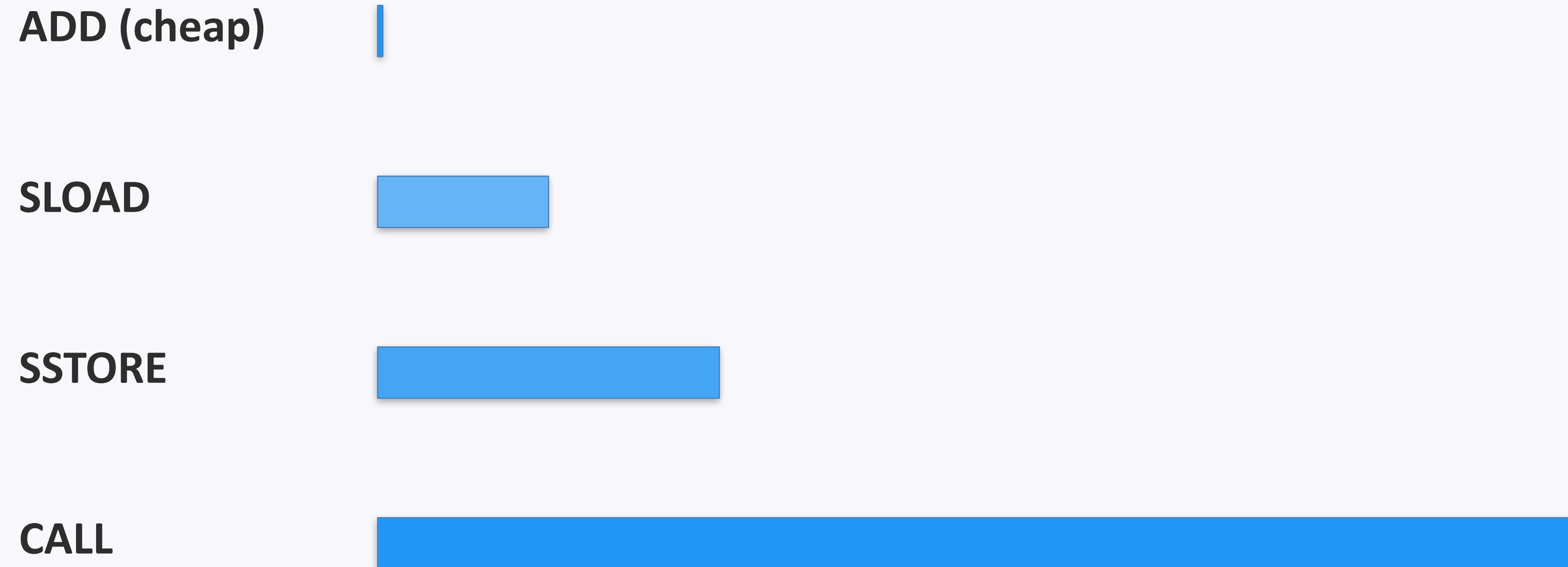
Amount (_value): 1000

The Resulting Calldata

[illegible]

```
0xa9059cbb = transfer(address, uint256)
```


Gas Costs for Different Operations



Storage writes (SSTORE) are the most expensive. Most gas optimisation focuses on reducing storage operations.

Gas Limits and Constraints

Current Block Gas Limit: 60 million gas

All transactions in a block must fit within this

Typical transaction: ~21,000 gas

Transaction Gas Limit:

Set by the user when sending transaction

If exceeded: execution stops and reverts

Gas Protection Against:

Infinite loops (execution halts when gas = 0)

DoS attacks (attackers pay for computation)

Excessive computation (economic ceiling)

How a Transaction Modifies State

1. User calls increment()

2. Validate & Load State

3. Execute Opcodes

4. Deduct Gas

5. Commit or Revert

Contract counter {

 Uint256 public counter;

 Function increment() public {

 counter += 1;

 }

}

Signature verified, sender has enough ETH

EVM loads contract storage, initialises memory

SLOAD current value, ADD 1, SSTORE new value

Gas deducted per opcode

If success: state saved. If fail: reverted

Real Example: increment() Function

Contract Code:

```
function increment() public {  
    count += 1;  
}
```

What Actually Happens:

1. SLOAD — Load current count from storage
2. PUSH 1 — Push constant 1 onto stack
3. ADD — Pop both, push sum (count + 1)
4. SSTORE — Store new value back to storage

Gas Cost Breakdown:

SLOAD: ~100 gas

ADD: 3 gas

SSTORE: ~20,000 gas ← most expensive!

Atomic Execution: All or Nothing

Transactions are atomic:

Either ALL changes succeed
Or the entire transaction reverts
No partial updates

Example: Execution Failure

```
require(msg.sender == owner, 'denied');
```

If require fails: state rolled back
Gas already spent is NOT refunded

Why Atomic?

Prevents inconsistent states
Simplifies smart contract logic

Module 2 Recap: The EVM

- ✓ EVM is a deterministic virtual machine
- ✓ Thousands of nodes compute identically
- ✓ Stack-based architecture (1024 items max)
- ✓ Memory is temporary, Storage is permanent
- ✓ Gas charged per opcode prevents abuse
- ✓ Transactions are state transition triggers
- ✓ Storage writes are the most expensive
- ✓ Atomic: All or nothing execution

Key Insight:

Ethereum's security comes from determinism, not trust

Homework:

Install Foundry:

Install the industry standard Solidity Framework:

- Go to <https://www.getfoundry.sh/> and follow the installation instructions for windows or Mac.